

BCHW Tensor Flattening and Reconstruction

Objective

This assignment implements the transformations

$$B \times C \times H \times W \rightarrow B \times (C \times H \times W) \rightarrow B \times C \times H \times W$$

using explicit indexing. The C++ implementation is in `tensor_flatten.cpp`, and the Python implementation is in `tensor_flatten.py`. Neither algorithm uses a library reshape/flatten operation to perform the required transformations. The Python program also uses NumPy reshape only for the separately labeled reference check, after the explicit implementation.

Index mapping

For BCHW row-major storage, the linear input offset is

$$\text{input_offset}(b, c, h, w) = b \times (C \times H \times W) + c \times (H \times W) + h \times W + w$$

For the two-dimensional tensor F with shape $B \times (CHW)$, the corresponding element is

$$\begin{aligned} F[b, k] &= I[b, c, h, w] \\ k &= c \times (H \times W) + h \times W + w \end{aligned}$$

To reconstruct, decode each flattened column index k :

$$\begin{aligned} c &= k // (H \times W) \\ \text{rem} &= k \% (H \times W) \\ h &= \text{rem} // W \\ w &= \text{rem} \% W \\ I_{\text{hat}}[b, c, h, w] &= F[b, k] \end{aligned}$$

The batch index b selects the row and is unchanged by either transformation.

Pseudocode

```
flatten(I, B, C, H, W):
    allocate F[B][C*H*W]
    for b in 0..B-1:
        for c in 0..C-1:
            for h in 0..H-1:
                for w in 0..W-1:
                    k = c*(H*W) + h*W + w
                    F[b][k] = I[b][c][h][w]
    return F

reconstruct(F, B, C, H, W):
    allocate I_hat[B][C][H][W]
    for b in 0..B-1:
```

```

for k in 0..C*H*W-1:
c = k // (H*W)
rem = k % (H*W)
h = rem // W
w = rem % W
I_hat[b][c][h][w] = F[b][k]
return I_hat

```

Manual example

Let B=1, C=2, H=2, W=3. The flattened row contains CHW = 12 elements. Using sample values $I[0,c,h,w] = 100c + 10h + w$:

BCHW element | Flattened index $k = c*6 + h*3 + w$ | Flattened value

```

BCHW element=`I[0,0,0,0]` | Flattened index `k = c*6 + h*3 + w`=0 | Flattened value=0
BCHW element=`I[0,0,1,2]` | Flattened index `k = c*6 + h*3 + w`=5 | Flattened value=12
BCHW element=`I[0,1,0,0]` | Flattened index `k = c*6 + h*3 + w`=6 | Flattened value=100
BCHW element=`I[0,1,1,2]` | Flattened index `k = c*6 + h*3 + w`=11 | Flattened value=112

```

Thus the complete flattened row is:

```
[0, 1, 2, 10, 11, 12, 100, 101, 102, 110, 111, 112]
```

Experiments and results

Both programs were run. The C++ program was compiled with `g++ -std=c++17 -Wall -Wextra -Wpedantic -O2`. The Python program ran with NumPy. Each implementation generated its own test input; the random Python tensor values and deterministic C++ values are not intended to be identical. We compared the reported error measures for the same six tensor shapes.

Dataset note: No MNIST or CIFAR dataset files are loaded by either program. For the first two rows, the programs generate synthetic tensors with dimensions matching a single-channel 28x28 MNIST image and a three-channel 32x32 CIFAR image. The remaining inputs are also synthetic feature maps.

Input | B | C | H | W | Elements | Emax | MAE

```

Synthetic (MNIST shape) | B=1 | C=1 | H=28 | W=28 | Elements=784 | Emax=0.00000 | MAE=0.00000
Synthetic (CIFAR shape) | B=1 | C=3 | H=32 | W=32 | Elements=3,072 | Emax=0.00000 | MAE=0.00000
Synthetic feature map 1 | B=2 | C=16 | H=16 | W=16 | Elements=8,192 | Emax=0.00000 | MAE=0.00000
Synthetic feature map 2 | B=2 | C=64 | H=16 | W=16 | Elements=32,768 | Emax=0.00000 | MAE=0.00000
Synthetic feature map 3 | B=2 | C=128 | H=16 | W=16 | Elements=65,536 | Emax=0.00000 | MAE=0.00000
Synthetic feature map 4 | B=2 | C=500 | H=16 | W=16 | Elements=256,000 | Emax=0.00000 | MAE=0.00000

```

The C++ and Python outputs both reported Emax = 0 and MAE = 0 for every case, including the manual example.

Error calculation and discussion

For every element, the programs calculate the absolute difference $\text{abs}(I - I_{\text{hat}})$. They report

```

Emax = maximum absolute difference over all elements
MAE = sum of absolute differences / number of elements

```

Zero errors are the expected result here. Flattening and reconstruction only copy each value to and from its corresponding index; they do not perform arithmetic on tensor values or change their data type. The explicit index formulas are inverse mappings, so every tested input element returns to its original position. This shows no reconstruction loss for these tests; it does not claim that every conceivable input or implementation is error-free.

Reproducing the experiments

With a C++17 compiler and NumPy installed:

```
g++ -std=c++17 -Wall -Wextra -Wpedantic -O2 tensor_flatten.cpp -o tensor_flatten.exe  
.\tensor_flatten.exe  
python tensor_flatten.py
```